S1. Tarea. Implementación de la estructura completa de una base de datos (DDL)

Integrantes del equipo:

Karen Julieth Castro Arias

Jorge Eliecer Mazo Uparela

Samuel Hincapié Pérez

Paula Andrea Morales Zapata

Ali Jose Arias Basto

Curso: Bases de Datos I – G100 (SD1006)

Oralia Cortes Grajales

Enlace al repositorio de GitHub:

https://github.com/Jorge-Mazo/bd1-redsocial-grupo6

Enlace al video:

https://drive.google.com/file/d/18c60pTNCFvnIF9zi7sgSlHs6DtULX1TM/view

18 de octubre de 2026

Introducción

En este trabajo llevamos a un sistema real el modelo lógico que entregamos en la Tarea 2 para la red social estudiantil pascualina. Ese modelo ya estaba normalizado hasta tercera forma normal y tenía 24 tablas repartidas en siete módulos: usuarios y datos académicos, mentorías, perfiles y preferencias, publicaciones e interacciones, recursos académicos, comunidades y eventos, y comunicación interna.

Elegimos PostgreSQL 16 como sistema de gestión de bases de datos y trabajamos con pgAdmin 4. El resultado es un script DDL que crea toda la estructura desde cero y que está versionado en el repositorio de Git del grupo.

En este informe explicamos cómo hicimos el paso del modelo lógico al físico: por qué elegimos PostgreSQL, cómo definimos los tipos y tamaños de cada campo, cómo implementamos las llaves primarias y foráneas, y qué acciones referenciales aplicamos en cada relación y por qué.

Selección del SGBD y justificación

Comparamos tres opciones: PostgreSQL, MySQL y Microsoft SQL Server. Al final nos quedamos con PostgreSQL.

Criterio	PostgreSQL	MySQL	SQL Server
Licencia	Código abierto, sin costo	Código abierto (edición Community)	Comercial; la edición Express es gratuita pero limitada
Estándar SQL	Muy alto (GENERATED ... AS IDENTITY, CHECK)	Bueno, con extensiones propias (AUTO_INCREMENT)	Dialecto propio (T-SQL)
Restricciones CHECK	Siempre aplicadas	Solo se aplican desde la versión 8.0.16	Completas
Fechas con zona horaria	timestamptz	Más limitado	datetimeoffset
Booleano	boolean nativo	Alias de tinyint	bit
Concurrencia	MVCC	MVCC en InnoDB	Bloqueos, con aislamiento por versiones opcional
Herramientas	pgAdmin 4	MySQL Workbench	SQL Server Management Studio

Por qué PostgreSQL. Nuestro modelo depende mucho de la integridad referencial: tiene 33 llaves foráneas, entre ellas una compuesta y varias reflexivas, y PostgreSQL las maneja de forma declarativa con todas las acciones referenciales. Además, los CHECK que

necesitamos (puntuación de 1 a 5, estados, roles, que nadie se siga a sí mismo) se aplican siempre, y timestamptz nos permite ordenar con precisión el muro, los comentarios y los mensajes. Es gratuito, multiplataforma y está entre los recursos de consulta de la unidad (Canepa, 2016; Hsu y Obe, 2015), así que todos los integrantes pudieron trabajar con el mismo entorno.

Desventajas. PostgreSQL pide más configuración y administración inicial que MySQL, y los hostings compartidos económicos suelen ofrecer MySQL antes. También tiene funciones avanzadas, como la partición o el ajuste fino del rendimiento, que exigen más conocimiento del que necesita este proyecto.

Por qué descartamos las otras dos. MySQL era una opción válida y más sencilla, pero el soporte tardío de CHECK en versiones antiguas y su manejo más limitado de fechas con zona horaria pesaron en contra. SQL Server es muy robusto, pero implica licencia o los límites de su edición gratuita, depende más del ecosistema Windows y usa T-SQL, que se aleja del estándar que vimos en la unidad.

Del modelo lógico al modelo físico

El modelo lógico define entidades, atributos, llaves y cardinalidades sin depender de ningún sistema. En el físico las decisiones son concretas, y esto fue lo que hicimos con cada elemento:

- Cada entidad se convirtió en una tabla (24 en total) y cada atributo en una columna con tipo, tamaño y regla de nulidad.
- Cada identificador pasó a ser una llave primaria INT GENERATED ALWAYS AS IDENTITY, con la restricción nombrada.
- Los atributos únicos (el correo institucional y los nombres de los catálogos) quedaron con restricciones UNIQUE.
- Las relaciones 1:N se resolvieron con una llave foránea en el lado “muchos”.
- La relación 1:1 entre estudiante y perfil social se resolvió con una FK que además es UNIQUE.
- Las relaciones N:M quedaron como tablas intermedias, con llave primaria compuesta por las dos llaves foráneas.
- Las relaciones reflexivas (seguir y enviar mensajes) tienen dos FK hacia estudiante, una por cada rol.
- Los atributos con valores fijos (estado, rol, nivel, tipo de reacción) se controlan con CHECK.
- Las reglas de integridad RI-01 a RI-09 se implementaron con NOT NULL, llaves foráneas, UNIQUE y CHECK.

Al pasar a físico tuvimos que ajustar algunas cosas que el modelo lógico no resolvía:

1. **Fechas con hora.** En el modelo lógico las fechas de publicación, comentario, reacción, recurso, valoración y mensaje eran DATE. Como en una red social importa la hora exacta, las pasamos a timestamptz. De hecho, fecha_envio del mensaje se describía como “fecha y hora” pero estaba definida como DATE.
2. **Restricciones de dominio nuevas.** Agregamos CHECK que antes no estaban escritos: semestre entre 1 y 10, creditos mayor que cero y los valores permitidos de nivel, estado, rol, asistencia, tipo de reacción y puntuación.
3. **Personas distintas.** Una sesión de mentoría exige mentor distinto de aprendiz, nadie puede seguirse a sí mismo y nadie puede enviarse mensajes a sí mismo.
4. **Llave foránea compuesta en las sesiones.** El modelo lógico tenía tres FK simples en sesion_mentoria: mentor, aprendiz y materia. En el físico, mentor y materia forman una FK compuesta hacia mentor_materia_habilitada. Así la base de datos garantiza que solo se agende una sesión con un mentor que esté habilitado en esa materia. La materia sigue relacionada, pero a través de la habilitación.
5. **Índices.** Creamos índices sobre las llaves foráneas que no quedan cubiertas por una PK compuesta o por un UNIQUE.

Reglas de nomenclatura y estándares

Aplicamos las reglas del libro Estructura, reglas y estándares de la unidad:

Elemento	Regla	Ejemplo
Tablas	Minúsculas, snake_case, singular y en español	estudiante, sesion_mentoria
Columnas	Minúsculas, snake_case, sin tildes	fecha_publicacion
Llaves	Prefijo codigo_ y nombre de la entidad	codigo_estudiante
FK con rol	El rol va en el nombre	codigo_mentor, codigo_seguido
Catálogos	Prefijo tipo_ cuando clasifican un atributo	tipo_grupo
Llave primaria	pk_<tabla>	pk_estudiante
Llave foránea	fk_<tabla>_<referencia>	fk_publicacion_estudiante
Único	uq_<tabla>_<campo>	uq_estudiante_correo
Check	chk_<tabla>_<campo>	chk_valoracion_rango
Índice	idx_<tabla>_<campo>	idx_publicacion_estudiante

En los diagramas y el informe de la Tarea 2 los nombres aparecían en mayúsculas solo por legibilidad. PostgreSQL convierte a minúsculas los identificadores sin comillas, así que en el modelo físico usamos minúsculas y evitamos las comillas dobles. Los valores de los CHECK también van sin tilde ('tecnico', 'asistio') para evitar errores de escritura al insertar datos.

Inventario de tablas

Módulo	Tablas
1. Usuarios y datos académicos	programa_academico, materia_asignatura, estudiante, perfil_social
2. Mentorías	mentor_materia_habilitada, sesion_mentoria
3. Perfiles y preferencias	interes, habilidad, estudiante_interes, estudiante_habilidad
4. Publicaciones e interacciones	tipo_publicacion, publicacion, seguidor_estudiante, comentario, reaccion
5. Recursos académicos	tipo_recurso, recurso_academico, valoracion_recurso
6. Comunidades y eventos	tipo_grupo, grupo, miembro_grupo, evento, asistencia_evento
7. Comunicación interna	mensaje

Son **24 tablas** en total.

Descripción de cada tabla

Cómo elegimos los tipos de dato

- **INT en lugar de BIGINT para las llaves.** Un INT llega a unos 2.147 millones de valores, mucho más de lo que necesita una comunidad universitaria. BIGINT ocuparía 8 bytes en lugar de 4 en cada llave primaria, en cada foránea y en cada índice, sin darnos ningún beneficio.
- **VARCHAR(n) en lugar de TEXT.** En PostgreSQL se almacenan igual y rinden igual, así que la diferencia es de diseño: el límite deja documentada la regla de negocio y evita contenidos desmedidos.
- **GENERATED ALWAYS AS IDENTITY en lugar de SERIAL.** IDENTITY es el mecanismo del estándar SQL, mientras que SERIAL es un atajo propio de PostgreSQL. Con ALWAYS nadie puede asignar un id a mano, así que la secuencia no se desincroniza.
- **timestamptz en lugar de timestamp o DATE.** Las acciones sociales necesitan fecha, hora y zona horaria para ordenarse bien.
- **DATE y TIME separados** en sesiones y eventos, porque son citas agendadas por día y hora y no instantes en que algo ocurrió. DATE solo se usa donde la hora no importa, como nacimiento, registro, unión o inscripción.
- **BOOLEAN** para leido, porque solo tiene dos estados y es un tipo nativo.
- **CHECK en lugar de ENUM**, para poder cambiar los valores permitidos sin crear tipos nuevos y dejarlos visibles en la definición de la tabla.
- **VARCHAR(255) para URLs y rutas**, porque solo guardamos la referencia al archivo y no el archivo.

En las tablas, *PK* es llave primaria, *FK* llave foránea, *UQ* único y *NN* no nulo. Todas las PK simples son INT GENERATED ALWAYS AS IDENTITY.

Módulo 1: usuarios y datos académicos

programa_academico es el catálogo de carreras.

Campo	Tipo	Restricciones	Justificación
codigo_programa	INT	PK	Identificador simple
nombre_programa	VARCHAR(100)	NN, UQ	Evita programas duplicados
nivel	VARCHAR(20)	NN, CHECK (‘tecnico’, ‘tecnologico’, ‘profesional’)	Dominio fijo de formación
descripcion	VARCHAR(255)	Opcional	Complemento no indispensable

materia_asignatura

Campo	Tipo	Restricciones	Justificación
codigo_materia	INT	PK	Identificador
nombre_materia	VARCHAR(100)	NN, UQ	Evita materias duplicadas
creditos	INT	NN, CHECK > 0	Número entero positivo

semestre	**INT**	**NN, CHECK 1 a 10**	**Ubica la materia en el plan**
codigo_programa	**INT**	**NN, FK**	**Toda materia pertenece a un programa**

estudiante es la entidad central**.**

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK	Dos estudiantes pueden compartir nombre, nunca el id
nombre	VARCHAR(50)	NN	Dato de identidad
apellido	VARCHAR(50)	NN	Dato de identidad
correo_institucional	VARCHAR(100)	NN, UQ	Identifica de forma única al estudiante
fecha_nacimiento	DATE	NN	Solo fecha, sin hora
semestre	INT	NN, CHECK 1 a 10	Sirve para decidir quién puede ser mentor
fecha_registro	DATE	NN, DEFAULT fecha actual	Se asigna sola al registrarse
codigo_programa	INT	NN, FK	RI-05: un único programa

perfil_social tiene relación 1:1 con **estudiante.**

Campo	Tipo	Restricciones	Justificación
codigo_perfil	INT	PK	Identificador propio
descripcion_personal	VARCHAR(255)	Opcional	Texto breve
area_estudio	VARCHAR(100)	NN	Requerido por el reto
fotografia	VARCHAR(255)	Opcional	Solo la URL o ruta
biografia	VARCHAR(500)	Opcional	Texto más largo
tipo_mentoria	VARCHAR(50)	Opcional	Solo aplica a mentores
codigo_estudiante	INT	NN, FK, UQ	El UNIQUE garantiza el 1:1 (RI-04)

Módulo 2: mentorías

mentor_materia_habilitada resuelve la relación N:M entre estudiante y materia**.**

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	**INT**	**PK compuesta, FK**	**Estudiante habilitado como mentor**
codigo_materia	**INT**	**PK compuesta, FK**	**Materia en la que se habilita**
fecha_habilitacion	**DATE**	**NN, DEFAULT fecha actual**	**Historial del proceso**
certificado	**VARCHAR(255)**	**Opcional**	**No siempre hay soporte formal**

sesion_mentoria

Campo	Tipo	Restricciones	Justificación
codigo_sesion	INT	PK	Cada cita es independiente
fecha_sesion	DATE	NN	Día agendado
hora_sesion	TIME	NN	Hora agendada
tema	VARCHAR(150)	NN	Contexto de la cita
estado	VARCHAR(20)	NN, DEFAULT 'programada', CHECK ('programada', 'realizada', 'cancelada')	Ciclo de vida de la sesión
codigo_mentor	INT	NN, FK compuesta	Parte de la FK (mentor, materia)
codigo_aprendiz	INT	NN, FK	Estudiante que recibe la mentoría
codigo_materia	INT	NN, FK compuesta	Parte de la FK (mentor, materia)

Tiene además la restricción CHECK (codigo_mentor <> codigo_aprendiz).

Módulo 3: perfiles y preferencias

interes y habilidad son catálogos con la misma estructura: una PK (codigo_interes o codigo_habilidad, INT), un nombre VARCHAR(80) obligatorio y único, y una descripción VARCHAR(255) opcional.

estudiante_interes (N:M)

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK compuesta, FK	Estudiante
codigo_interes	INT	PK compuesta, FK	Interés
fecha_asociacion	DATE	DEFAULT fecha actual	Dato propio de la relación

estudiante_habilidad (N:M)

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK compuesta, FK	Estudiante
codigo_habilidad	INT	PK compuesta, FK	Habilidad
nivel	VARCHAR(20)	Opcional, CHECK (‘basico’, ‘medio’, ‘avanzado’)	Nivel de dominio de esa pareja

Módulo 4: publicaciones e interacciones

tipo_publicacion es un catálogo: codigo_tipo_publicacion (INT, PK), nombre_tipo (VARCHAR(50), NN, UQ) y descripcion (VARCHAR(255), opcional).

publicacion

Campo	Tipo	Restricciones	Justificación
codigo_publicacion	INT	PK	Identificador
contenido	VARCHAR(500)	NN	Texto del muro
fecha_publicacion	timestamptz	NN, DEFAULT hora actual	Orden cronológico preciso
archivo_adjunto	VARCHAR(255)	Opcional	No toda publicación lo tiene
codigo_estudiante	INT	NN, FK	RI-01: toda publicación tiene autor
codigo_tipo_publicacion	INT	NN, FK	Clasificación

seguidor_estudiante es la N:M reflexiva.

Campo	Tipo	Restricciones	Justificación
codigo_seguidor	INT	PK compuesta, FK	Quien sigue
codigo_seguido	INT	PK compuesta, FK	A quien se sigue
fecha_seguimiento	DATE	NN, DEFAULT fecha actual	Cuándo empezó

Tiene la restricción CHECK (codigo_seguidor <> codigo_seguido).

comentario lleva PK propia.

Campo	Tipo	Restricciones	Justificación
codigo_comentario	INT	PK	Un estudiante puede comentar varias veces la misma publicación
contenido	VARCHAR(500)	NN	Texto
fecha_comentario	timestamptz	NN, DEFAULT hora actual	Orden cronológico
codigo_estudiante	INT	NN, FK	Autor
codigo_publicacion	INT	NN, FK	Publicación comentada

reaccion (N:M)

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK compuesta, FK	Quien reacciona
codigo_publicacion	INT	PK compuesta, FK	Publicación
tipo_reaccion	VARCHAR(20)	NN, CHECK (‘me gusta’, ‘me encanta’, ‘me divierte’, ‘me asombra’)	Valores predefinidos
fecha_reaccion	timestamptz	NN, DEFAULT hora actual	Registro temporal

La PK compuesta limita a una reacción por estudiante y publicación.

Módulo 5: recursos académicos

tipo_recurso es un catálogo: codigo_tipo_recurso (INT, PK), nombre_tipo (VARCHAR(50), NN, UQ) y descripcion (VARCHAR(255), opcional).

recurso_academico

Campo	Tipo	Restricciones	Justificación
codigo_recurso	INT	PK	Identificador
titulo	VARCHAR(150)	NN	Para identificarlo en listados
descripcion	VARCHAR(500)	Opcional	Texto libre
url_archivo	VARCHAR(255)	NN	Todo recurso debe tener dónde encontrarse
fecha_subida	timestamptz	NN, DEFAULT hora actual	Orden cronológico
codigo_estudiante	INT	NN, FK	Autor (RI-08)
codigo_materia	INT	NN, FK	Materia asociada (RI-08)
codigo_tipo_recurso	INT	NN, FK	Clasificación

valoracion_recurso (N:M)

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK compuesta, FK	Quien valora
codigo_recurso	INT	PK compuesta, FK	Recurso valorado
puntuacion	INT	NN, CHECK 1 a 5	Escala simple
comentario	VARCHAR(255)	Opcional	No toda valoración lo lleva
fecha_valoracion	timestamptz	NN, DEFAULT hora actual	Orden

Módulo 6: comunidades y eventos

tipo_grupo es un catálogo: codigo_tipo_grupo (INT, PK), nombre_tipo (VARCHAR(50), NN, UQ) y descripcion (VARCHAR(255), opcional).

grupo

Campo	Tipo	Restricciones	Justificación
codigo_grupo	INT	PK	Identificador
nombre_grupo	VARCHAR(100)	NN, UQ	Evita nombres repetidos
descripcion	VARCHAR(500)	Opcional	Texto libre
fecha_creacion	DATE	NN, DEFAULT fecha actual	Antigüedad del grupo
codigo_estudiante_creador	INT	NN, FK	RI-02: todo grupo tiene creador
codigo_tipo_grupo	INT	NN, FK	Clasificación

evento

Campo	Tipo	Restricciones	Justificación
codigo_evento	INT	PK	Identificador
nombre_evento	VARCHAR(100)	NN	Identificación
fecha_evento	DATE	NN	Día
hora_evento	TIME	NN	Hora
lugar	VARCHAR(150)	NN	Dónde se realiza
estado	VARCHAR(20)	NN, DEFAULT 'programado', CHECK ('programado', 'realizado', 'cancelado')	Ciclo de vida
descripcion	VARCHAR(500)	Opcional	Texto libre
codigo_estudiante_creador	INT	NN, FK	RI-03: todo evento tiene creador

miembro_grupo (N:M)

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK compuesta, FK	Miembro
codigo_grupo	INT	PK compuesta, FK	Grupo
fecha_union	DATE	NN, DEFAULT fecha actual	Antigüedad en el grupo

rol	VARCHAR(20)	NN, DEFAULT ‘miembro’, CHECK (‘miembro’, ‘moderador’)	Nivel de permisos

asistencia_evento (N:M)

Campo	Tipo	Restricciones	Justificación
codigo_estudiante	INT	PK compuesta, FK	Inscrito
codigo_evento	INT	PK compuesta, FK	Evento
fecha_inscripcion	DATE	NN, DEFAULT fecha actual	Control de inscripciones
estado_asistencia	VARCHAR(20)	NN, DEFAULT ‘inscrito’, CHECK (‘inscrito’, ‘asistio’, ‘no asistio’)	Distingue inscripción de asistencia real

Módulo 7: comunicación interna

mensaje es una relación reflexiva con PK propia.

Campo	Tipo	Restricciones	Justificación
codigo_mensaje	INT	PK	Un mismo par de estudiantes intercambia muchos mensajes

contenido	VARCHAR(500)	NN	Texto
fecha_envio	timestamptz	NN, DEFAULT hora actual	Orden de la conversación
leido	BOOLEAN	NN, DEFAULT false	Control de mensajes no leídos
codigo_emisor	INT	NN, FK	Quien envía
codigo_receptor	INT	NN, FK	Quien recibe

Tiene la restricción CHECK (codigo_emisor <> codigo_receptor).

Implementación de las relaciones y acciones referenciales

Criterios que aplicamos

Usamos ON DELETE CASCADE cuando el registro hijo no tiene sentido sin su padre, como un perfil sin estudiante, una reacción sin publicación o cualquier fila de una tabla intermedia. Usamos ON DELETE RESTRICT cuando borrar el padre destruiría información que debe conservarse o que otros usan: los catálogos en uso, los creadores de grupos y eventos, y el historial de sesiones de mentoría.

ON UPDATE CASCADE lo aplicamos en todas las llaves foráneas como política uniforme y defensiva, para que un cambio de llave se propague. Como las PK son IDENTITY y no se espera que cambien, en la práctica esta acción casi nunca se ejecuta.

Tabla de relaciones

La tabla padre es la referenciada y la hija es la que contiene la llave foránea.

#	Padre	Hija	FK en la hija	ON DELETE	ON UPDATE	Justificación
1	programa_academico	materia_asignatura	codigo_programa	RESTRICT	CASCADE	No se borra un programa con materias
2	programa_academico	estudiante	codigo_programa	RESTRICT	CASCADE	No se borra un programa con estudiantes (RI-05)
3	estudiante	perfil_social	codigo_estudiante (UQ)	CASCADE	CASCADE	El perfil no existe sin el estudiante (RI-04)

4	estudiante	mentor_materia_habilitada	codigo_estudiante	CASCADE	CASCADE	La habilitación pierde sentido sin el estudiante
5	materia_asignatura	mentor_materia_habilitada	codigo_materia	CASCADE	CASCADE	La habilitación pierde sentido sin la materia
6	mentor_materia_habilitada	sesion_mentoria	(codigo_mentor, codigo_materia)	RESTRICT	CASCADE	Se conserva el historial; no se borra una habilitación con sesiones (RI-07)
7	estudiante	sesion_mentoria	codigo_aprendiz	RESTRICT	CASCADE	No se borra al aprendiz con sesiones (RI-07)
8	estudiante	estudiante_interes	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
9	interes	estudiante_interes	codigo_interes	CASCADE	CASCADE	Tabla intermedia
10	estudiante	estudiante_habilidad	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
11	habilidad	estudiante_habilidad	codigo_habilidad	CASCADE	CASCADE	Tabla intermedia
12	estudiante	publicacion	codigo_estudiante	CASCADE	CASCADE	Si se elimina al autor se

						eliminan sus publicaciones (RI-01)
13	tipo_publicacion	publicacion	codigo_tipo_publicacion	RESTRICT	CASCADE	No se borra un tipo en uso
14	estudiante	seguidor_estudiante	codigo_seguidor	CASCADE	CASCADE	El seguimiento depende de ambos estudiantes
15	estudiante	seguidor_estudiante	codigo_seguido	CASCADE	CASCADE	Ídem
16	estudiante	comentario	codigo_estudiante	CASCADE	CASCADE	Se eliminan los comentarios del autor
17	publicacion	comentario	codigo_publicacion	CASCADE	CASCADE	Un comentario no existe sin su publicación
18	estudiante	reaccion	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
19	publicacion	reaccion	codigo_publicacion	CASCADE	CASCADE	Tabla intermedia
20	estudiante	recurso_academico	codigo_estudiante	CASCADE	CASCADE	Se eliminan los recursos del autor (RI-08)

21	materia_asignatura	recurso_academico	codigo_materia	RESTRICT	CASCADE	No se borra una materia con recursos (RI-08)
22	tipo_recurso	recurso_academico	codigo_tipo_recurso	RESTRICT	CASCADE	No se borra un tipo en uso
23	estudiante	valoracion_recurso	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
24	recurso_academico	valoracion_recurso	codigo_recurso	CASCADE	CASCADE	Una valoración no existe sin el recurso
25	estudiante	grupo	codigo_estudiante_creador	RESTRICT	CASCADE	No se deja un grupo sin creador (RI-02)
26	tipo_grupo	grupo	codigo_tipo_grupo	RESTRICT	CASCADE	No se borra un tipo en uso
27	estudiante	evento	codigo_estudiante_creador	RESTRICT	CASCADE	No se deja un evento sin creador (RI-03)
28	estudiante	miembro_grupo	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
29	grupo	miembro_grupo	codigo_grupo	CASCADE	CASCADE	Tabla intermedia
30	estudiante	asistencia_evento	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia

31	evento	asistencia_ evento	codigo_evento	CASCADE	CASCADE	Tabla intermedia
32	estudiante	mensaje	codigo_emisor	CASCADE	CASCADE	Se eliminan los mensajes enviados (RI-09)
33	estudiante	mensaje	codigo_receptor	CASCADE	CASCADE	Se eliminan los mensajes recibidos (RI-09)

Decisiones que vale la pena destacar

Si se intenta borrar un estudiante que creó grupos o eventos, o que tiene sesiones de mentoría, el borrado queda bloqueado por RESTRICT. Si no tiene nada de eso, se eliminan en cascada su perfil, publicaciones, comentarios, reacciones, recursos y mensajes. Es coherente con la integridad, pero es una decisión fuerte: en un sistema real consideraríamos un borrado lógico, con una columna como activo, para no perder información.

Con los mensajes usamos CASCADE en emisor y receptor para cumplir RI-09, y la consecuencia es que al eliminar a un estudiante el otro participante pierde esa conversación. Algo parecido pasa con los recursos académicos: el CASCADE desde el autor elimina recursos que otros pudieron haber valorado. Lo aceptamos porque el recurso depende de su autor, aunque una mejora futura sería conservarlo.

Con las materias, la habilitación de mentores se elimina en cascada, pero el borrado se bloquea si hay sesiones o recursos asociados, para proteger el historial académico.

Estructura del script DDL

El script se llama ddl_red_social_estudiantil_pascualina.sql y está en el repositorio en la ruta tarea3/ddl/ddl_red_social_estudiantil_pascualina.sql. Está organizado en cinco partes:

1. **Cabecera.** Describe el proyecto, el SGBD, los integrantes y las notas de diseño: nombres en minúsculas, uso de timestamptz y orden de creación.
2. **Limpieza previa.** Una serie de DROP TABLE IF EXISTS ... CASCADE, en orden inverso a las dependencias, para poder reejecutar el script desde cero.
3. **Los siete módulos de CREATE TABLE**, en el orden que exigen las dependencias: primero los catálogos y las tablas padre, después las hijas. Por ejemplo, mentor_materia_habilitada se crea antes que sesion_mentoria porque esta tiene una FK compuesta hacia ella. Todas las restricciones van nombradas con los prefijos pk_, uq_, chk_ y fk_.
4. **Índices sobre las llaves foráneas.** Son 24 índices idx_. No duplicamos los que ya existen por una PK o un UNIQUE, y idx_sesion_mentor_materia cubre la FK compuesta.
5. **Comentarios** (COMMENT ON) en las 24 tablas y en las columnas con significado especial, que quedan visibles en la herramienta gráfica.

Evidencia de ejecución

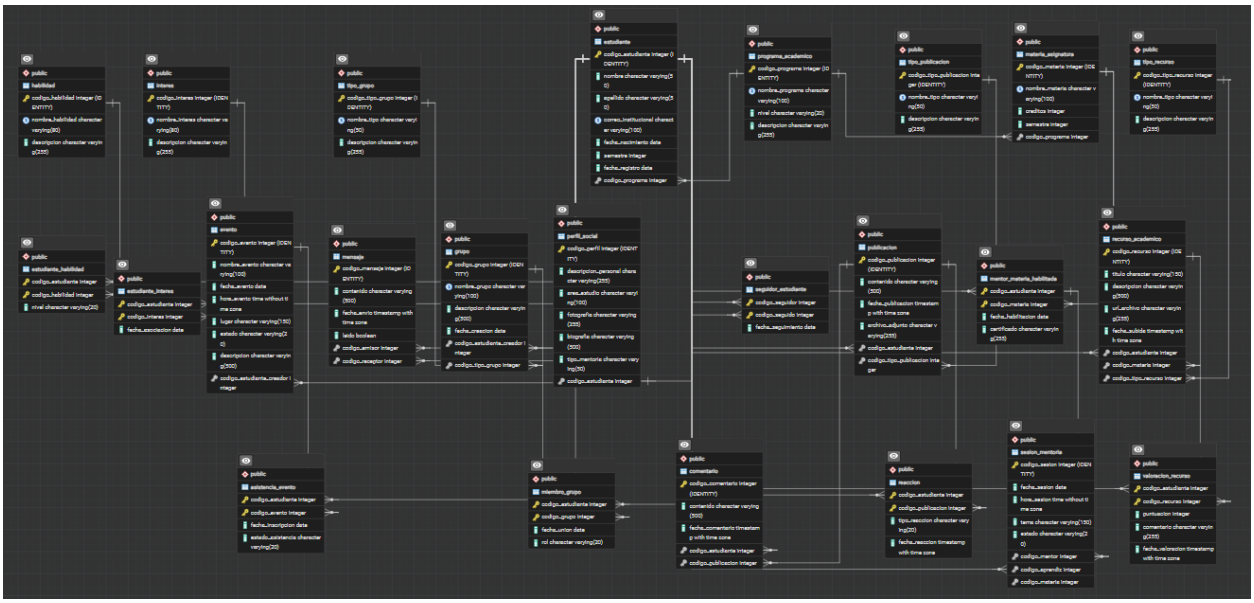
Árbol de tablas en pgAdmin 4



Ejecución completa del script DDL en PostgreSQL 16, sin errores.

```
Query returned successfully in 477 msec.
Total rows:    Query complete 00:00:00.477
```

Diagrama entidad-relación generado desde la base de datos implementada.



Conclusiones individuales

Jorge Eliecer Mazo Uparela

Mi mayor dificultad en esta tarea fue entender cómo las decisiones del modelo lógico se convertían en restricciones concretas del SGBD, sobre todo en las relaciones. En la Tarea 1 me costó el rompimiento de las relaciones N:M y en la Tarea 2 revisar dependencias parciales y transitivas. Al pasar al físico entendí por qué esas reglas importan: una FK compuesta en sesion_mentoria hacia mentor_materia_habilitada hace que la propia base de datos impida crear una sesión con un mentor no habilitado en esa materia, algo que en el modelo lógico solo estaba escrito en el papel. Aprendí a justificar ON DELETE y ON UPDATE según el tipo de relación, y a reconocer que CASCADE no siempre es la opción por defecto. Me quedó la duda de si ON UPDATE CASCADE es realmente necesario cuando las PK son IDENTITY, y la dejé documentada como medida defensiva. Como mejora futura, propondría un borrado lógico de estudiantes con una columna activo para no perder el historial de publicaciones y mensajes.

Samuel Hincapié Pérez

Mi aporte se centró en revisar que el script fuera coherente con el modelo lógico, tabla por tabla. Al comparar los dos modelos encontré diferencias que no había visto en la Tarea 2, como las fechas de interacción que estaban definidas como DATE pero en realidad necesitaban hora, o los CHECK que no estaban escritos. Aprendí a justificar el tipo de dato por su uso real y no por costumbre: timestamptz para el muro y los mensajes, DATE para nacimiento y registro, BOOLEAN para leido. La mayor dificultad fue entender por qué la tabla comentario necesita PK propia y reaccion PK compuesta, cuando ambas relacionan estudiante y publicación; comprendí que depende de si la acción se puede repetir muchas veces o solo una vez por combinación. Si repitiera el trabajo, haría primero una lista de las reglas de

integridad RI-01 a RI-09 y las iría marcando tabla por tabla, para no dejar ninguna sin implementar.

Ali Jose Arias Basto

Mi parte fue el módulo de recursos académicos. En el modelo físico la dificultad principal fue decidir entre CASCADE y RESTRICT cuando un recurso ya tiene valoraciones de otros estudiantes. Si se borra al autor, ¿se borran las valoraciones de otros? Optamos por CASCADE porque el recurso depende del autor, pero reconozco que es discutible y lo dejé documentado como decisión a destacar. Aprendí a justificar VARCHAR(n) frente a TEXT y INT frente a BIGINT por razones concretas de dominio y rendimiento, y no por costumbre. También entendí por qué GENERATED ALWAYS AS IDENTITY es preferible a SERIAL: es el estándar SQL y evita que la secuencia se desincronice si alguien inserta un id a mano. Para mejorar, propondría incluir datos semilla en el script para que la base quede con ejemplos listos para probar.

Karen Julieth Castro Arias

Trabajé el módulo de publicaciones e interacciones. En la Tarea 1 me había costado la relación reflexiva de seguir, y en esta tarea entendí por qué seguidor_estudiante necesita PK compuesta y no una PK propia como publicacion. La PK compuesta (codigo_seguidor, codigo_seguido) impide por diseño que un estudiante siga dos veces al mismo compañero, y el CHECK que agregamos (codigo_seguidor <> codigo_seguido) impide que alguien se siga a sí mismo. Entendí que son dos reglas distintas y que cada una la resuelve una restricción diferente. También aprendí que comentario lleva PK propia porque un estudiante puede comentar muchas veces la misma publicación en momentos distintos, mientras que reaccion lleva PK compuesta porque solo puede reaccionar una vez por publicación. Mi mayor dificultad fue entender por qué la base rechaza una sesión con un mentor no habilitado en la materia; al

probarlo en pgAdmin lo vi funcionar y ahí comprendí el valor de la FK compuesta. Como mejora futura, agregaría una tabla de auditoría para registrar cambios en las publicaciones, algo que en una red social real sería importante.

Paula Andrea Morales Zapata

Trabajé sobre todo en la revisión de las relaciones y las acciones referenciales. Entendí que ON DELETE no es un detalle: CASCADE y RESTRICT cambian qué pasa con los datos de otras personas. Por ejemplo, si se borra un estudiante que creó un grupo, el RESTRICT bloquea el borrado y protege la integridad de RI-02; pero si no tiene nada creado, sus comentarios y reacciones se van en cascada. Eso me generó dudas sobre si es correcto que se pierdan las conversaciones de otras personas cuando alguien se va de la plataforma. Aprendí que probar el script con inserciones reales ayuda a encontrar errores que no se ven leyendo el código, como la FK compuesta que rechaza sesiones con mentores no habilitados. Como mejora futura, propondría conservar los grupos y eventos cuando su creador se elimina, usando ON DELETE SET NULL en codigo_estudiante_creador. Eso exigiría quitarle el NOT NULL a esas columnas, y por tanto flexibilizar las reglas RI-02 y RI-03, que hoy obligan a que todo grupo y todo evento tenga creador. Habría que decidir si esa regla vale más que conservar el registro, y esa es una decisión de negocio, no solo técnica.